

Cours 4

Une animation, du notebook à un programme en ligne de commande

1re année géomatique

Objectifs de la séance

Écrire un projet Python qui fabrique une courte vidéo animée.

Le rendu est un dossier versionné avec git, qui contient un script callable en ligne de commande : `python montre.py` OU `python tourbillon.py`.

Partie	Ce qu'on fait	Durée
Présentation	les deux TD, les outils, les étapes	10 min
A · Exécuter le notebook	dans VS Code, terminal Git Bash : créer l'environnement <code>animation</code> , y lancer JupyterLab, exécuter le notebook	35 min
B · Du notebook au programme	dans VS Code : le script construit par fonctionnalités (une image, une série, la vidéo), une branche git par fonctionnalité	70 min
Fin	montrer sa vidéo et son <code>git log</code>	5 min

Deux TD au choix

Les deux TD fabriquent une vidéo de quelques secondes. Chaque image de la vidéo correspond à une valeur d'un paramètre : l'heure pour la montre, l'angle de torsion pour le tourbillon.

	4a · La montre du Lapin blanc	4b · La Vague en tourbillon
Dossier	cours4/4a_montre/	cours4/4b_tourbillon/
Une image	un cadran, deux aiguilles, le Lapin, l'heure	La Grande Vague , tordue par <code>-swirl</code> , l'angle
Ce qui varie	l'heure, d'une minute par image	l'angle, de 15° par image, aller et retour
Ce que Python calcule	la position des aiguilles, par <code>sin</code> et <code>cos</code>	la liste des angles
Les fonctionnalités (partie B)	une image : <code>--heure 10:05</code> ; une série : <code>--minutes</code> ; la vidéo : <code>--video</code>	une image : <code>vague.jpg --angle 90</code> ; une série : <code>--maximum</code> ; la vidéo : <code>--video</code>

Dans le TD 4a, chaque image est dessinée à partir de formes simples ; dans le TD 4b, chaque image est une transformation d'une image existante.

Des outils en ligne de commande

ImageMagick (`magick`) et `ffmpeg` sont des programmes en ligne de commande, comme `git` (cours 2) et `pandoc` (cours 3). Le script Python les lance avec `subprocess.run` : une fois par image pour `magick`, une fois à la fin pour `ffmpeg`. Python automatise ainsi toutes les étapes de la fabrication de la vidéo.

```
magick -size 640x480 xc:#fbf7ee -draw "..." img_0001.png  
magick petite.png -swirl 90 -extent 640x480 img_0001.png  
ffmpeg -framerate 12 -i img_%04d.png montre.mp4
```

4a : une image unie, puis des formes

4b : l'image, tordue de 90 degrés

12 images par seconde ; %04d : un numéro à quatre chiffres

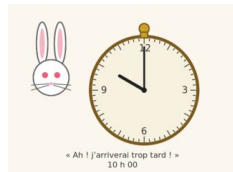
Pendant le développement, chaque étape se termine par un `commit git` : le dépôt garde une version qui fonctionne à chaque étape.

Le programme montre.py, étape par étape

Pour chaque minute, Python calcule l'angle et l'extrémité des deux aiguilles, puis lance `magick`, qui dessine l'image. `ffmpeg` assemble les 120 images.



Images écrites à l'étape 4 dans `sortie/images/`, puis la vidéo écrite à l'étape 5 :



10 h 00



10 h 30



11 h 00



11 h 30



11 h 59

→ montre.mp4

Le programme tourbillon.py, étape par étape

Python calcule la liste des angles de torsion, puis, pour chaque angle, lance `magick`, qui tord l'image réduite. `ffmpeg` assemble les 49 images.



Images écrites à l'étape 4 dans `sortie/images/`, puis la vidéo écrite à l'étape 5 :



0°



90°



180°



270°



360°

→ `tourbillon.mp4`

A • Exécuter le notebook

Un environnement conda décrit par `environment.yml`,
JupyterLab lancé depuis cet environnement

L'environnement animation

`environment.yml` liste ce que le projet demande. `conda env create` crée l'environnement à partir de ce fichier ; `conda activate` le rend actif dans le terminal.

`depart/environment.yml`

```
name: animation
channels:
  - conda-forge
dependencies:
  - python=3.12
  - jupyterlab
  - imagemagick
  - ffmpeg
```

Dans le terminal Git Bash, dans `depart/`

```
conda env create -f environment.yml
conda env list → une ligne animation
conda activate animation → (animation)
magick -version, ffmpeg -version
```

Guide, étapes A2 et A3. Avant la première commande, une fois par poste : `conda` rendu disponible dans Git Bash. La création télécharge les paquets : plusieurs minutes.

JupyterLab lancé depuis l'environnement

Le notebook appelle `magick` et `ffmpeg` : il les cherche dans le `PATH` du terminal qui a lancé JupyterLab. JupyterLab se lance donc depuis le terminal Git Bash de VS Code, l'environnement `animation` actif.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	copier <code>depart/notebook/<nom>.ipynb</code> dans <code>travail/</code>	<code>travail/<nom>.ipynb</code>
2	<code>cd travail, puis jupyter lab</code>	le navigateur s'ouvre sur <code>travail/</code>
3	ouvrir le notebook, exécuter la section 1	trois chemins qui contiennent <code>envs\animation</code>
4	exécuter les sections suivantes, une à une	une image d'essai par section, puis la vidéo

Attention. Lancé depuis Anaconda Navigator, JupyterLab tourne dans `base` : la section 1 affiche `magick : None`.

La montre du Lapin blanc

Créer l'environnement animation, exécuter le notebook montre.ipynb, puis construire le script montre.py
fonctionnalité par fonctionnalité : une image, une série, la vidéo ; une branche git par fonctionnalité

cours4/4a_montre/ ≈ 105'

A1 à A3 · L'environnement animation

Dans VS Code, avec un terminal Git Bash : rendre `conda` disponible, créer l'environnement `animation`, l'activer, vérifier les deux outils.

	Ce qu'il faut faire	Ce que vous devez obtenir
A1	décompresser <code>info01-cours4.zip</code> sur le Bureau ; ouvrir <code>cours4/4a_montre/</code> dans VS Code ; terminal Git Bash	<code>pwd</code> se termine par <code>cours4/4a_montre</code>
A2	une fois par poste : <code>source /c/ProgramData/anaconda3/ etc/profile.d/conda.sh</code> , <code>conda init bash</code> , nouveau terminal ; puis dans <code>depart/</code> : <code>conda env create -f environment.yml</code>	<code>invite (base)</code> ; <code>conda env list</code> affiche <code>animation</code>
A3	<code>conda activate animation</code> ; <code>magick -version</code> , <code>ffmpeg -version</code>	l' <code>invite</code> commence par <code>(animation)</code> ; les deux versions s'affichent

Guide, étapes A1 à A3. La création télécharge les paquets : plusieurs minutes.

A4 · Le notebook

JupyterLab est lancé depuis le terminal Git Bash, l'environnement `animation` actif : c'est ainsi que le notebook trouve `magick` et `ffmpeg`.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	copier <code>depart/notebook/montre.ipynb</code> dans <code>travail/</code> ; cd <code>travail</code> , jupyter lab	JupyterLab s'ouvre sur <code>travail/</code>
2	exécuter le notebook, section par section	section 1 : trois chemins dans <code>envs\animation</code> ; section 6 : la vidéo

Guide, étape A4. Ne pas lancer JupyterLab depuis Navigator : `magick` n'y est pas trouvé.

B0 · Le projet et le dépôt git

Dans un second terminal Git Bash (le premier fait tourner JupyterLab) : `conda activate animation`, puis le dossier du projet.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	<code>mkdir travail/montre ; y copier</code> <code>depart/environment.yml</code> ; <code>cd</code> dans le dossier	l'invite se termine par <code>travail/montre</code>
2	<code>git init</code> ; <code>echo "sortie/" > .gitignore</code> ; un <code>README.md</code> d'une ligne	<code>git status</code> : les fichiers non suivis, sans <code>depart/</code>
3	<code>git add .</code> ; <code>git commit -m "Le projet : environnement, .gitignore et README"</code>	<code>git log --oneline</code> : une ligne

La suite construit le programme fonctionnalité par fonctionnalité, une branche git par fonctionnalité.

B1 · Une image, sur la branche une-image

Le programme dessine une image. Il a dès le départ une fonction `main` et lit ses options avec `argparse`.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	<code>git checkout -b une-image</code>	<code>git branch : * une-image</code>
2	<code>montre.py</code> : les imports, les outils, les fonctions de dessin du notebook ; un commit	<code>python montre.py</code> n'affiche rien, sans erreur
3	la fonction <code>main</code> et les options ; <code>git commit -am "..."</code>	<code>python montre.py --heure 10:05</code> écrit <code>sortie/montre_1005.png</code>
4	<code>git checkout master ; git merge une-image</code>	Fast-forward ; trois commits

B2 · Une série d'images, sur la branche serie

Le programme écrit une image par valeur du paramètre dans `sortie/images/`. Sans la nouvelle option, il écrit toujours une image seule.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	<code>git checkout -b serie</code>	<code>git branch : * serie</code>
2	la fonction <code>serie</code> , sous les fonctions de dessin ; un commit	<code>python montre.py --help</code> fonctionne comme avant
3	l'option <code>--minutes</code> dans <code>main</code> ; un commit	<code>python montre.py --heure 10:00 --minutes 120</code> remplit <code>sortie/images/</code>
4	<code>git checkout master ; git merge serie</code>	Fast-forward ; cinq commits

B3 · La vidéo, et un commit de fusion

ffmpeg assemble la série en vidéo. Pendant ce travail, un commit est fait sur `master` : la fusion crée un commit de fusion.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	<code>git checkout -b video</code> ; la fonction <code>assembler</code> , les options <code>--video</code> et <code>--cadence</code> ; un commit	<code>python montre.py --heure 10:00 --minutes 120 --video</code> écrit <code>sortie/montre.mp4</code>
2	<code>git checkout master</code> ; compléter le README ; un commit ; <code>git checkout video</code>	le README de <code>master</code> décrit une image et la série
3	<code>import shutil</code> ; la fonction <code>nettoyer</code> et l'option <code>--nettoyer</code> ; un commit	<code>sortie/images/</code> est supprimé après la vidéo
4	<code>git checkout master</code> ; <code>git merge --no-edit video</code>	Merge made by the 'ort' strategy. ; neuf commits

`git log --oneline --graph --all` dessine les deux branches et le commit de fusion.

B4 et B5 · README, commande installée

Le README décrit les trois fonctionnalités. L'étape B5, facultative, fait du script une commande montre.

	Ce qu'il faut faire	Ce que vous devez obtenir
B4	<code>copier</code> <code>depart/modeles/README.md</code> sur le README et le compléter ; <code>git commit -am "README complet"</code>	<code>git log --oneline</code> : dix lignes
B5	<code>git mv montre.py src/montre.py</code> ; <code>copier</code> <code>depart/modeles/pyproject.toml</code> ; <code>pip install -e .</code>	Successfully installed montre-0.1
	<code>montre --help</code> depuis un autre dossier ; <code>git add .</code> ; commit	l'aide s'affiche ; onze lignes dans <code>git log</code>

Le rendu : le dossier `travail/montre/` et son historique `git`.

La Vague en tourbillon

Créer l'environnement animation, exécuter le notebook tourbillon.ipynb, puis construire le script tourbillon.py fonctionnalité par fonctionnalité : une image, une série, la vidéo ; une branche git par fonctionnalité

cours4/4b_tourbillon/ ≈ 105'

A1 à A3 · L'environnement animation

Dans VS Code, avec un terminal Git Bash : rendre `conda` disponible, créer l'environnement `animation`, l'activer, vérifier les deux outils.

	Ce qu'il faut faire	Ce que vous devez obtenir
A1	décompresser <code>info01-cours4.zip</code> sur le Bureau ; ouvrir <code>cours4/4b_tourbillon/</code> dans VS Code ; terminal Git Bash	<code>pwd</code> se termine par <code>cours4/4b_tourbillon</code>
A2	une fois par poste : <code>source /c/ProgramData/anaconda3/</code> <code>etc/profile.d/conda.sh</code> , <code>conda init bash</code> , nouveau terminal ; puis dans <code>depart/</code> : <code>conda env create -f</code> <code>environment.yml</code>	<code>invite (base)</code> ; <code>conda env list</code> affiche <code>animation</code>
A3	<code>conda activate animation</code> ; <code>magick -version</code> , <code>ffmpeg -version</code>	l' <code>invite</code> commence par <code>(animation)</code> ; les deux versions s'affichent

Guide, étapes A1 à A3. La création télécharge les paquets : plusieurs minutes.

A4 · Le notebook

JupyterLab est lancé depuis le terminal Git Bash, l'environnement `animation` actif : c'est ainsi que le notebook trouve `magick` et `ffmpeg`.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	copier <code>depart/notebook/tourbillon.ipynb</code> dans <code>travail/</code> ; <code>cd travail, jupyter lab</code>	JupyterLab s'ouvre sur <code>travail/</code>
2	exécuter le notebook, section par section	section 1 : trois chemins dans <code>envs\animation</code> ; section 6 : la vidéo

Guide, étape A4. Ne pas lancer JupyterLab depuis Navigator : `magick` n'y est pas trouvé.

B0 · Le projet et le dépôt git

Dans un second terminal Git Bash (le premier fait tourner JupyterLab) : `conda activate animation`, puis le dossier du projet.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	<code>mkdir travail/tourbillon ; y copier depart/environment.yml et depart/vague.jpg ; cd</code> dans le dossier	l'invite se termine par <code>travail/tourbillon</code>
2	<code>git init ; echo "sortie/" > .gitignore ; un README.md d'une ligne</code>	<code>git status</code> : les fichiers non suivis, sans <code>depart/</code>
3	<code>git add . ; git commit -m "Le projet : environnement, .gitignore et README"</code>	<code>git log --oneline</code> : une ligne

La suite construit le programme fonctionnalité par fonctionnalité, une branche git par fonctionnalité.

B1 · Une image, sur la branche une-image

Le programme dessine une image. Il a dès le départ une fonction `main` et lit ses options avec `argparse`.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	<code>git checkout -b une-image</code>	<code>git branch : * une-image</code>
2	<code>tourbillon.py</code> : les imports, les outils, les fonctions de dessin du notebook ; un commit	<code>python tourbillon.py</code> n'affiche rien, sans erreur
3	la fonction <code>main</code> et les options ; <code>git commit -am "..."</code>	<code>python tourbillon.py vague.jpg --angle 90</code> écrit <code>sortie/tourbillon_090.png</code>
4	<code>git checkout master ; git merge une-image</code>	Fast-forward ; trois commits

B2 · Une série d'images, sur la branche serie

Le programme écrit une image par valeur du paramètre dans `sortie/images/`. Sans la nouvelle option, il écrit toujours une image seule.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	<code>git checkout -b serie</code>	<code>git branch : * serie</code>
2	les fonctions <code>angles</code> et <code>serie</code> , sous les fonctions de dessin ; un commit	<code>python tourbillon.py --help</code> fonctionne comme avant
3	l'option <code>--maximum</code> dans <code>main</code> ; un commit	<code>python tourbillon.py vague.jpg --maximum 360</code> remplit <code>sortie/images/</code>
4	<code>git checkout master ; git merge serie</code>	Fast-forward ; cinq commits

B3 · La vidéo, et un commit de fusion

ffmpeg assemble la série en vidéo. Pendant ce travail, un commit est fait sur `master` : la fusion crée un commit de fusion.

	Ce qu'il faut faire	Ce que vous devez obtenir
1	<code>git checkout -b video</code> ; la fonction assembler, les options <code>--video</code> et <code>--cadence</code> ; un commit	<code>python tourbillon.py vague.jpg --maximum 360 --video</code> écrit <code>sortie/tourbillon.mp4</code>
2	<code>git checkout master</code> ; compléter le README ; un commit ; <code>git checkout video</code>	le README de <code>master</code> décrit une image et la série
3	<code>import shutil</code> ; la fonction nettoyer et l'option <code>--nettoyer</code> ; un commit	<code>sortie/images/</code> est supprimé après la vidéo
4	<code>git checkout master</code> ; <code>git merge --no-edit video</code>	Merge made by the 'ort' strategy. ; neuf commits

`git log --oneline --graph --all` dessine les deux branches et le commit de fusion.

B4 et B5 · README, commande installée

Le README décrit les trois fonctionnalités. L'étape B5, facultative, fait du script une commande `tourbillon`.

	Ce qu'il faut faire	Ce que vous devez obtenir
B4	<code>copier</code> <code>depart/modeles/README.md</code> sur le README et le compléter ; <code>git commit -am "README complet"</code>	<code>git log --oneline</code> : dix lignes
B5	<code>git mv tourbillon.py src/tourbillon.py</code> ; <code>copier</code> <code>depart/modeles/pyproject.toml</code> ; <code>pip install -e .</code>	Successfully installed tourbillon-0.1
	<code>tourbillon --help</code> depuis un autre dossier ; <code>git add .</code> ; <code>commit</code>	l'aide s'affiche ; onze lignes dans <code>git log</code>

Le rendu : le dossier `travail/tourbillon/` et son historique `git`.

Ce qu'on rend

Le dossier du projet, avec son historique git : le code, `environment.yml` et le README, qui permettent de refaire la vidéo sur un autre poste.

Commande	Ce qu'elle doit montrer
<code>git log --oneline --graph --all</code>	dix commits (onze avec B5), dont un commit de fusion
<code>git status</code>	« rien à valider » ; <code>sortie/</code> n'est pas listé
<code>python <nom>.py --help</code>	les options des trois fonctionnalités, et leur aide
<code>sortie/<nom>.mp4</code>	la vidéo, qui s'ouvre par un double-clic